Distributed Cross-Entropy Optimization for Max-Cut of Large Graphs on HPC Systems

Andrew Baumes
Jeremy Goldberger
Liam Olausson
Erik Umble
Rensselaer Polytechnic Institute
Troy, New York, USA

Abstract

The Max-Cut problem is a fundamental NP-complete combinatorial optimization challenge with applications in network design, logistics, and data analysis. While various approximation algorithms exists, achieving high-quality solutions for large scale graphs requires significant computational resources. In this paper, we present a distributed, high-performance implementation of the Cross-Entropy (CE) method for approximating Max-Cut on large graphs.

Our approach utilizes a hybrid MPI+CUDA architecture following a distributed "island model." Each MPI rank manages an independent CE process on an NVIDIA V100 GPU, where we leverage massively parallel kernels to sample and evaluate millions of candidate partitions per second per rank. We implement bitwise optimizations to accelerate the running time of the kernel. To reduce local optima stagnation while maintaining global search, we employ a periodic synchronization strategy that shares promising probability states across the computing cluster.

We evaluate our implementation on the AiMOS supercomputer using the G-Set and Biq Mac benchmarks and with random Erdős-Rényi graphs. Our results demonstrate excellent scalability, with near-linear speedup in strong scaling tests up to 16 GPUs. Experimentally, our algorithm reaches within 1.12% of the best-known solutions for the G-Set benchmark. This demonstrates a promising approach for approximating the Max-Cut problem on HPC systems.

CCS Concepts

• **Computing methodologies** → **Massively parallel algorithms**; • **Theory of computation** → *Stochastic approximation*; *Massively parallel algorithms.*

Keywords

Massively parallel algorithms, Cross-Entropy Method, Max-Cut Problem, GPU acceleration, Distributed optimization, Combinatorial optimization, MPI, CUDA, Strong and weak scaling

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PCP '26, Troy, NY, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Andrew Baumes, Jeremy Goldberger, Liam Olausson, and Erik Umble. 2026. Distributed Cross-Entropy Optimization for Max-Cut of Large Graphs on HPC Systems. In *Proceedings of Parallel Programming and Computing (PCP '26).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Max-Cut is a well known problem from graph theory. Given a graph $G = (V, E)$ the task is to find a subset $S \subset V$ which maximizes the number of edges from S to $\overline{S} = V \setminus S$. This problem has applications in other problems, including network optimization, logistics, and data clustering. Although Max-Cut is an NP-Complete problem, many approaches have been applied to obtain approximate solutions. In this paper, we aim to evaluate a High Performance Computing (HPC)-based approach to approximating Max-Cut.

Famously, Geomans and Williams showed a semidefinite programming approach obtains a result within 87.856% of the optimal in polynomial time [5]. More recently, Quantum Approximate Optimization Algorithm (QAOA) based methods have been used [9].

Quantum inspired evolutionary algorithms are another approach for combinatorial optimization problems. It has been used for solving the knapsack problem and suggested for other similar problems [6]. We considered this approach but decided against it due to the complexity of implementing it on an HPC system.

The Cross-Entropy (CE) method is another powerful approach for approximating combinatorial optimization problems [11]. In CE, many samples are taken from a probability distribution to iteratively update the distribution towards optimal solutions. It has been used successfully applied for Max-Cut [8] and its highly parallel nature makes it a strong candidate for an HPC-based implementation.

In this paper we will use a similar approach as Laguna et. al [8] but view the algorithm from a massively parallel perspective, evaluating its approximation power and scaling performance on an HPC system. We present a MPI+CUDA CE algorithm for Max-Cut that scales to highly parallel computing systems. Approximation power is evaluated on the well-known G-Set [7] and Biq Mac [10] benchmarks while strong and weak scaling are measured on random graphs of controlled sizes.

2 Cross-Entropy Max-Cut Algorithm

The Cross-Entropy (CE) method is an iterative random algorithm that uses two steps [2]:

(1) Sample from a probability distribution over the possible solutions
(2) Use the samples to modify the probability distribution to bias towards better samples in the next iteration

Oftentimes, many samples are taken from the distribution in each iteration and used to update the distribution. This can be parallelized as samples may be taken and evaluated independently in step (1).

2.1 Mathematical Model

Suppose G is a graph $G = (V(G), E(G))$ with n vertices. A cut is specified by a set of the vertices. In the undirected, unweighted formulation of Max-Cut, the value of a cut is simply the number of edges having one endpoint in the cut set. We formalize this with a Boolean variable to label each vertex. Vertices labeled 1 are in the cut set and the value of the cut is the number of edges whose endpoint labels sum to 1.

We formulate this in the CE method. Label each vertex $i \in V(G)$ with and independent Bernoulli random variable x_i having probability p_i. Initially, set $p_i = 0.5$ for all i as we have no knowledge of successful cuts on the graph. We denote $\mathbf{x} = (x_1, x_2, \ldots x_n)$ and $\mathbf{p} = (p_1, p_2, \ldots p_n)$. For a particular cut sample $\mathbf{x}$, the value of the cut is

$$f(\mathbf{x}) = \sum_{(i,j)\in E(G)} x_i \oplus x_j$$

where $\oplus$ denotes exclusive or.

Sample $\mathbf{x}$ N times, evaluating each with $f(x)$ and keeping the top k samples $\mathbf{x}_1^*, \mathbf{x}_2^*, \ldots, \mathbf{x}_\mathbf{k}^*$. We update probabilities using the CE update rule with smoothing

$$\mathbf{p_{t+1}} = (1-\alpha)\mathbf{p_t} + \alpha \frac{1}{k} \sum_{i=1}^{k} \mathbf{x}_\mathbf{i}^*$$

We use $k = 16$ and $\alpha = 0.1$. Probabilities are clipped to $[0.01, 0.99]$ to avoid allow for randomness in each iteration to avoid collapse to a particular decision for the vertex.

2.2 Algorithm Design

Our approach formulates the Max-Cut problem as a probabilistic optimization task, solved using a distributed Cross-Entropy Method CE described above. Rather than maintaining a single candidate solution, the algorithm maintains a probability vector, designated as qstate, which models the likelihood of each vertex belonging to one side of the bipartite cut.

The algorithm operates on a distributed island model [1] architecture using the message passing interface (MPI). Multiple independent processes execute the CE simultaneously, each exploring the search space with its own random seed [11]. To prevent the islands from completely diverging and to accelerate convergence on a global maximum, the algorithm enforces periodic synchronization governed by the communication_delay parameter. At these intervals, all nodes perform a global reduction to identify the highest cut achieved across the entire cluster. The node possessing the global best broadcasts its qstate to all other nodes, effectively pulling the distributed swarm toward the most promising region of the search space.

The core algorithm relies on two primary routines, the serial host control loop and the GPU evaluation kernel. The host process is responsible for initializing the probability state, orchestrating the GPU kernel executions, applying the CE update rules, and handling all MPI communications. The state vector qstate is initially completely uniform, 0.5 for all vertices, representing maximum entropy. After each batch of parallel GPU evaluations, the CPU extracts the top $K = 16$ samples. The probability of each vertex is then updated toward the mean of these elite samples using an exponentially weighted moving average with a learning rate $\alpha = 0.1$.To maintain exploration and prevent the algorithm from prematurely collapsing into a local optimum, the probabilities are limited to the range of $[0.01, 0.99]$.

Algorithm 1 Distributed Host (CPU) Control Loop

Require: Graph $G = (V, E)$, *iterations*, *subiterations*, *communication_delay*, $\alpha = 0.1$, $K = 16$
Ensure: *max_cut*, optimal *qstate*

```
Initialize MPI environment (rank, npes)
Initialize random seed using base_seed + rank
for each v ∈ V do
    qstate[v] ← 0.5
end for
max_cut ← 0
for i = 0 to iterations − 1 do
    # Send sampling and evaluation to GPU
    fast_cut_kernel(subiterations, G, qstate, max_cuts, out_states)
    # State Selection
    top_K_indices ← indices of the K largest values in max_cuts
    # Get Update
    for each j ∈ V do
        sum ← 0
        for k ∈ top_K_indices do
            sum ← sum + out_states[k].bit[j]
        end for
        mean ← sum/K
        qstate[j] ← (1.0 − α) × qstate[j] + (α × mean)
        qstate[j] ← max(0.01, min(0.99, qstate[j]))
    end for
    # Island Synchronization
    if i mod communication_delay == 0 then
        local_best ← Maximum cut found by this rank
        global_best, best_rank ← MPI_Allreduce(local_best, MPI_MAXLOC)
        max_cut ← global_best
        MPI_Bcast(qstate, best_rank) Broadcast best state
    end if
end for
```

The fast_cut kernel executes the probabilistic sampling and objective function evaluation. The graph's adjacency matrix and candidate partitions are compressed into uint64_t bit-arrays graph_var_t to improve performance. The objective function calculates the cut cost directly via bitwise AND operations between the partition

state and the adjacency matrix, leveraging the hardware __popcll, population count, intrinsic to instantly sum the crossing edges.

Algorithm 2 GPU Kernel (`fast_cut`)

Require: Graph G, $qstate$, $subiterations$
Ensure: max_cuts, out_states

```
thread_id ← Get global thread index
our_best_cut ← 0
best_state_local ← ∅
for iter = 0 to subiterations − 1 do
    new_state ← ∅ Initialize empty bit-array
    # Generate partition sample on qstate probabilities
    for each v ∈ V do
        rand_val ← Random float in [0.0, 1.0)
        if rand_val < qstate[v] then
            Set bit v in new_state to 1
        else
            Set bit v in new_state to 0
        end if
    end for
    # Bitwise Evaluation
    current_cut ← 0
    for each v ∈ V do
        if bit v in new_state == 0 then
            crossing_edges ← new_state & G.adjacency_row[v]
            current_cut ← current_cut + popcount(crossing_edges)
        end if
    end for
    # Track local best
    if current_cut > our_best_cut then
        our_best_cut ← current_cut
        best_state_local ← new_state
    end if
end for
max_cuts[thread_id] ← our_best_cut
out_states[thread_id] ← best_state_local
```

2.3 Communication Strategies

The algorithm uses MPI to coordinate the exploration execution across the GPUs. During the initialization phase, MPI facilitates the propagation of the graph's adjacency structure from the root process to all participating computing nodes. To mitigate communication bottlenecks the frequency of inter-node synchronization is explicitly parameterized via a the communication_delay variable. This hyperparameter dictates the exact number of iterations each MPI rank computes independently on its assigned GPU hardware before global state reconciliation.

At each designated synchronization interval, the system performs a parallel global reduction operation utilizing MPI_Allreduce coupled with the MPI_MAXLOC operator. This operation identifies both the highest objective value, the maximum cut, achieved across the entire network and the specific process rank responsible for its discovery. Following this reduction step, the process holding the global maximum broadcasts its optimal probability state vector, qstate, to all other ranks utilizing MPI_Bcast.

By parameterizing the synchronization intervals rather than communicating continuously, the algorithm allows for a tuning of the exploration/exploitation threshold. A higher communication_delay promotes broader, divergent exploration of the search space on local nodes, while a lower delay forces a more accurate global consensus and a quicker exploitation of promising probability states.

2.4 Implementation Details

2.4.1 CUDA Acceleration. We implemented our algorithm in CUDA to get the performance of massive parallelization. Each CUDA thread generates a random partition using the current probability state. For example, if a vertex has a 0.8 value in the probability state, then it has an 80% chance of being included in the generated partition. After generating this partition, the CUDA thread calculates the cut with this partition and stores the cut and partition in output memory. It repeats this some number of times (sub-iterations), only storing the output if the cut is better than the best this thread has found.

The main loop runs the CUDA kernel with approximately 100,000 threads and then finds the maximum 16 cuts across all generated partitions. Each vertex in the probability state is updated using the proportion of the maximum 16 cuts that contain this vertex. The CUDA kernel is then run again with this updated probability state. This repeats for some number of iterations.

2.4.2 MPI Coordination. We use MPI for inter-process coordination and collaboration in searching for the best cut. Our program input is fairly small (size n^2), so we choose to have only the root process read the input file and broadcast the full graph to the other ranks. As described above, each process maintains its own probability distribution state that is sampled from in the CUDA kernels that it launches. Each process also maintains the value of the best cut it has seen so far. Periodically, processes synchronize and update their state to be the state from the process that has found the best cut.

We use a *communication_delay* parameter for the number of iterations between each of these process synchronization events. Increasing the delay allows for more diverse exploration and lower communication overhead, but may also slow convergence to an optimal cut. A separate parameter, *subiterations*, controls the number of samples that each thread makes from the probability distribution before reporting back to the CPU. We use these parameters to tune the behavior when running on multiple nodes so that search space is explored somewhat similarly irrespective of the computing scale.

3 Experimental Setup

3.1 Hardware and Software Stack

Experiments are run on the Artificial Intelligence Multiprocessing Optimized System (AiMOS) that is part of the DCS cluster at the Center for Computational Innovations (CCI) of the Rensselaer Polytechnic Institute (RPI). Each compute node contains 2x IBMPOWER9 CPUs clocked at 3.15 GHz with 512 GiB RAM. Each node also has 4x NVIDIA Tesla V100 GPUs with 16 GiB RAM each.

An EDR infiniBand high-speed interconnect relays communication between nodes. Code is compiled using the CUDA compiler (V11.0.221) and the xlC-based MPI compiler (V16.1). It is run using MPI (V10.4).

3.2 Datasets and Benchmarks

We use the G-Set benchmark [7] for max-cut to evaluate the approximation performance of our algorithm compared to best known cuts found in the graph dataset. This dataset includes weighted and unweighted graphs of the following sizes (in number of vertices): 800, 1000, 2000, 3000, 5000, 7000, 10000, 20000. We benchmark our algorithm on G-Set to tune the number of iterations we use to be a number that brings us close to finding an optimal cut for graphs of sizes represented in the benchmark.

While the G-Set benchmark is useful for evaluating the utility of our algorithm, it does not lend itself well to a weak scaling experiment. Since the input size of a graph grows quadratically in the number of vertices, we want a range of graphs that have a power of $\sqrt{2}$ vertices. We construct a collection of such graphs using the Erdős–Rényi $G(n, p)$ random graph model [3]. These are constructed using a fixed edge probability of 0.5 for very dense graphs and values of n in $\{\sqrt{2^{16}}, \sqrt{2^{17}}, \sqrt{2^{18}}, \sqrt{2^{29}}, \sqrt{2^{20}}\}$.

We also use the Biq Mac library [12] of Max-Cut graphs to benchmark our algorithm. This dataset primarily contains small graphs, ranging from 20-500 vertices, that are used to test exact solution solvers for max-cut. We use them, in addition to the G-set graphs, to test the correctness of our algorithm for small graphs.

3.3 Methodology

We set up our strong and weak scaling experiments to run the same number of total cut calculations in each run and between each global and local sync. To do this, we decrease the number of sub-iterations as we increase the number of GPUs/MPI ranks (we use one MPI rank per GPU). When we double the GPU count, we halve the number of sub-iterations to keep the total number of checks constant. The problem size is defined by the total number of vertices in the graph.

For strong scaling, we increase the number MPI ranks while keeping the problem size consistent. For weak scaling, we increase the number of MPI ranks alongside increasing the number of vertices in our graph.

These different setups are used to test the scalability of our algorithm as we increase the graph size.

4 Performance Evaluation

4.1 Approximation Performance Benchmark

We run our algorithm on a selection of graphs from the G-Set and Biq Mac datasets. Specifically, we run many iterations of our algorithms on G1 and G2, and shorter iteration counts on G1, G43, G44. G22, G25, and G55. We also run the smaller Biq Mac graphs g05_60 and g05_100.

All runs in Table 1 used 1 node, 4 ranks, 256 iterations, and 4 subiterations. Our algorithm performs modestly well on the G1 graph, reaching a final max-cut of 11457, a result only 1.46% different than the best known. The execution time increases as the graph size increases, and the accuracy of the algorithm decreases.

Table 1: Results of running the algorithm on the G-Set

Name	Size	Time (seconds)	Result	Best Known	% Diff
G1	800	85.97	11457	11624	1.46%
G43	1000	108.94	6473	6660	2.89%
G44	1000	109.18	6486	6650	2.53%
G22	2000	636.36	12810	13359	4.28 %
G26	2000	646.96	12751	13328	4.53 %
G55	5000	4049.84	8697	10299	18.42 %

We also run extended runs of our algorithm on the G1 and G2 graph, which are each 800 nodes. The algorithm does not converge on either case, instead approaching a cut below the best known cut. The G1 run, graphed in Figure 1, approached a value of 11,495, approximately 1.12% lower than the best known value of 11,624 [4]. Similarly, by running on G2, graphed in Figure 2 we reach a similar value of 11,485, also approximately 1.12% lower than the best known value of 11,620 [4]. These runs show that the algorithm quickly converges on a value close to the maximum, and then approached the best known max-cut more slowly for the remainder. The algorithm is also subject to finding local maxima from which it can not escape from.

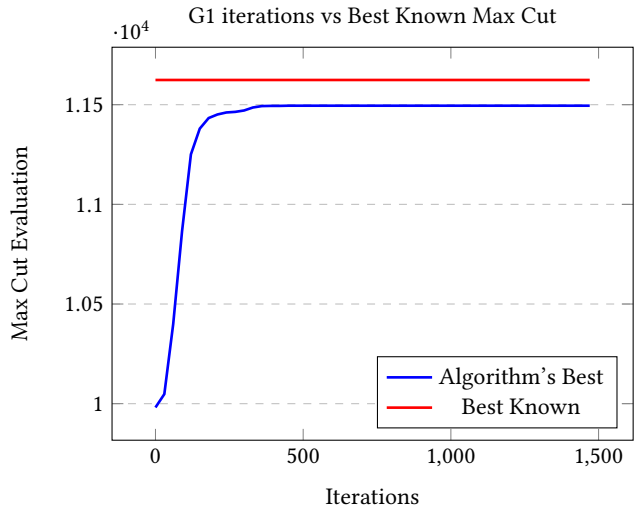

Figure 1: Performance of the algorithm on the G1 graph. It stagnates after 450 iterations.

The algorithm performs significantly more accurately on the smaller graphs from Biq Mac. Figure 3 shows the accuracy of our algorithm on the 60 node graph of g05_60. We reach the known maximum after only 16 iterations, resulting in a 0% difference.

Similarly, on the 100 node graph of g05_100 shown in Figure 4, the algorithm reaches a maximum of 1427 out of a best possible Max-Cut of 1430 after only 34 iterations, giving us a 0.21% difference. From this, it is easy to observe that our algorithm is susceptible to local maxima traps even at small graph sizes.

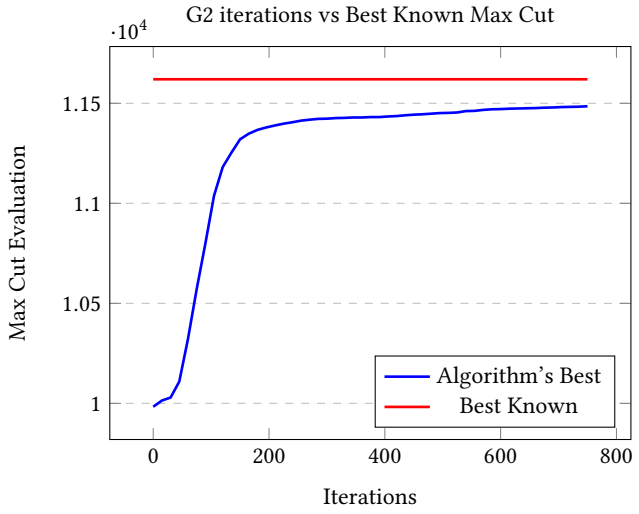

Figure 2: Performance of the algorithm on the G2 graph.

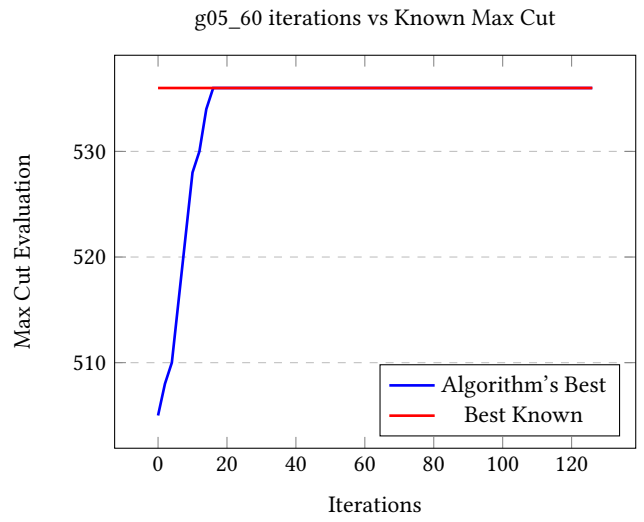

Figure 3: Performance of the algorithm on the Biq Mac g05_60 graph. It stagnates after just 16 iterations at 530, exactly the know max cut.

4.2 Strong Scaling

This strong scaling analysis shows the algorithm's capability to solve a problem of fixed size faster as additional computational resources are introduced. For this study, the graph size was held constant at 1024 vertices with 128 iterations, while the number of ranks was progressively doubled from a baseline of 2 up to 16. The runs demonstrate performance gains at each step, approaching ideal linear scaling but exhibiting the expected slight diminishing returns as communication overhead increases. Calculating the speedup for each step by dividing the execution time of the previous rank count by the execution time of the current rank count, we observe a speedup of ≈ 1.89× when moving from 2 to 4 ranks.

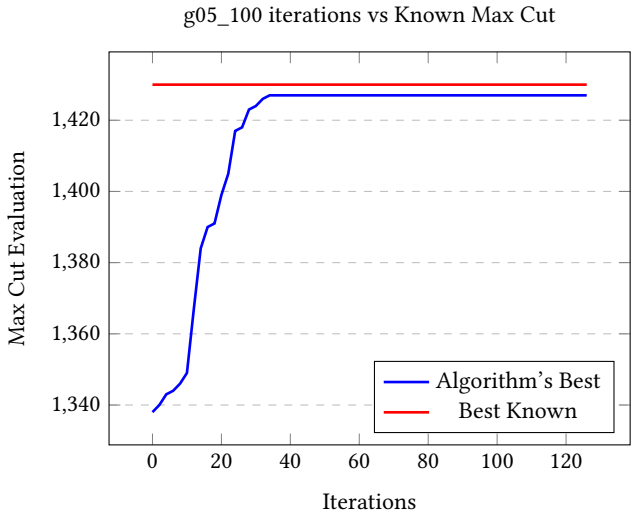

Figure 4: Performance of the algorithm on the Biq Mac g05_100 graph. It stagnates after just 34 iterations at 1427, within 1% of the known max cut of 1430.

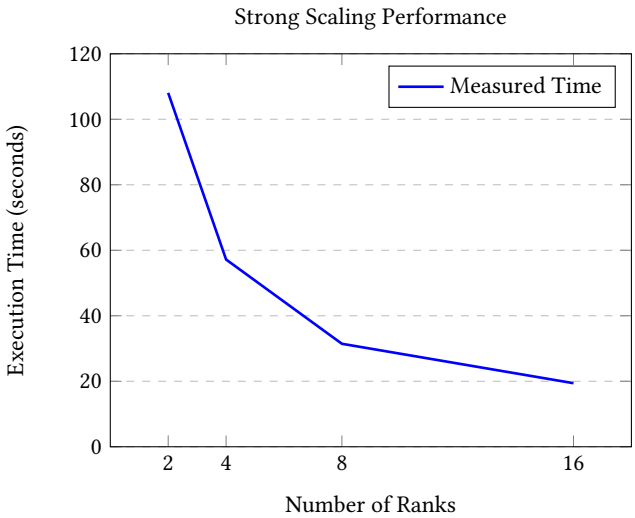

Figure 5: Strong scaling for a 1024 vertex graph. The total computational workload is kept constant across runs at 128 iterations, while the number of sub-iterations per rank halves as the number of ranks doubles.

Transitioning from 4 to 8 ranks yields a speedup of ≈ 1.82×. Finally, increasing the resources from 8 to 16 ranks results in a speedup of ≈ 1.62×. Compared to the 2-rank baseline, the 16-rank configuration achieves an overall speedup of ≈ 5.57×, meaning that the parallelization of this algorithm is quite good but not perfect.

4.3 Weak Scaling

The weak scaling analysis for the algorithm was quite interesting. In a perfectly ideal weak scaling scenario, the execution time would

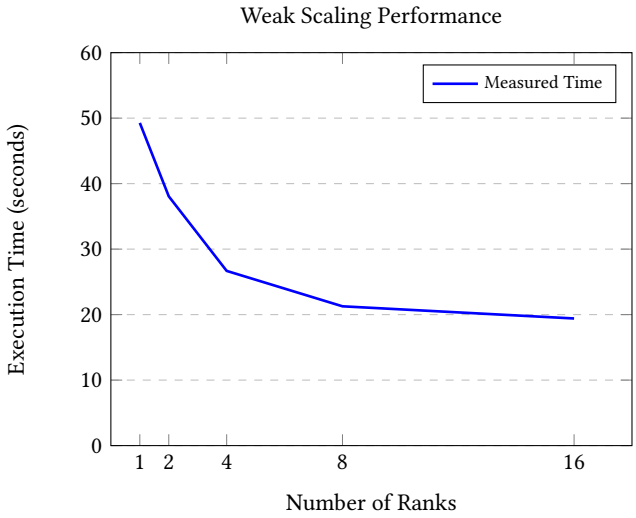

Figure 6: Weak scaling performance. As the number of compute ranks increases from 1 to 16, the problem size (graph size) scales up accordingly from 256 to 1024 vertices. Subiterations are scaled down inversely to maintain a consistent global partition generation rate per iteration.

remain entirely flat, however, our data shows that the execution time actually decreases as the graph size scales from 256 vertices on 1 rank to 1024 vertices on 16 ranks. Calculating the speedup yields a $\approx 1.29\times$ speedup from 1 to 2 ranks, a $\approx 1.43\times$ speedup from 2 to 4 ranks, a $\approx 1.25\times$ speedup from 4 to 8 ranks, and a $\approx 1.10\times$ speedup from 8 to 16 ranks. The overall time reduction from 1 rank to 16 ranks equates to a $\approx 2.54\times$ speedup. It is important to note that the weak scaling is sub-linear due to the scaling of RNG optimizations, which will be comprehensively explained later in the Discussion and Analysis section.

5 Analysis and Discussion

The algorithm performed extremely well on small graphs from the Biq Mac library, converging to within 1% of the known solution in just a few iterations as shown in Figure 3 and Figure 4 . Heuristic approaches such as the cross-entropy method are much better suited for optimization problems that are too large for traditional solvers to exhaustively check for the exact solution. We use graphs present in the G-Set benchmark, such as G1 with 800 nodes, to stress the algorithm and compare our solutions with known best results.

Convergence for large graphs takes significantly longer than for small graphs, and the Max-Cut is not found. As discussed in section 4.1, on G1, the algorithm reaches its maximum and then stagnated. It is expected that the algorithm would asymptotically approach the best known value. As the best known cut increases, the probability that the algorithm finds a new best cut decreases, and thus the rate of improvement decreases. However, failing to improve at all on G1 is indicative of the algorithm getting stuck in a local maxima. Our algorithm is disincentivized from exploring new states that are highly distinct from the current best known state, since they are improbable, and thus is unable to continue exploring new states that may be more optimal.

The algorithm scaled with compute extremely well. Execution time decreased nearly linearly as more ranks were added and we maintained very high GPU utilization, demonstrating excellent strong scaling. However, our weak scaling evaluation seen in Figure 6 showed an unexpected decrease in execution time rather than the theoretical constant flatline. This is due to a mismatch between the two primary workloads within the kernel, the cut evaluation and the RNG. The cut evaluation is an $O(V^2)$ operation as seen in algorithm 2, whereas generating the initial state population via the RNG is merely an $O(V)$ operation. To maintain a constant workload per rank for the core $O(V^2)$ evaluation, the graph size, V, was scaled by $\sqrt{N}$, where N is the number of ranks, while the subiterations loop, I, was inversely scaled by $\frac{1}{N}$. As intended, this kept the cut evaluation workload per thread constant, $I \cdot V^2 \propto \left(\frac{1}{N}\right)(\sqrt{N})^2 = O(1)$. However, applying this exact same $\frac{1}{N}$ subiteration reduction to the 1-dimensional RNG workload resulted in a mathematically shrinking workload, $I \cdot V \propto \left(\frac{1}{N}\right)\sqrt{N} = O\left(\frac{1}{\sqrt{N}}\right)$. Because managing the curand state requires expensive global memory reads and writes, it accounts for a large portion of the kernel's execution time at small graph sizes. Consequently, as N increased, the $O\left(\frac{1}{\sqrt{N}}\right)$ decay in RNG operations per thread dominated the performance profile, artificially reducing the total execution time despite the cut evaluation scaling correctly. This does mean that the the cut evaluation function in algorithm algorithm 2 will only start to become a significant portion of the runtime as graph sizes get larger.

6 Summary and Future Work

We employ the cross-entropy method for optimizing the Max-Cut of large graphs to moderate success. The algorithm can reach within 2% of the known best in under 200 seconds for 800 node G-Set graphs on the AiMOS system with 16 ranks. Despite this success, there is still room for significant improvement in many areas.

The algorithm was subject to stagnating in local maxima before it had reached the best known maximum cut. Further work on this algorithm would involve ways of escaping local maxima, allowing the algorithm to continue optimizing effectively even after it had reached a maximum.

Additionally, the algorithm is mathematically straightforward, and does not include optimizations such as avoiding rechecking the same state multiple times. With a large number of threads, as qstates converge on likely values, it is likely that single states are checked multiple times. Statistical analysis on the repetition of checked states would guide optimizations in this area, increasing the number of states that can be checked overall and improving the search space of the algorithm. We can also explore methods of escaping local maxima.

This project explored the performance of our cross-entropy algorithm. Other methods exist for approximating Max-Cut, including semi-definite programming, simulated annealing, K-opt, and genetic algorithms. Benchmarking our algorithm against these other approximation algorithms, as well as exploring the results of combining them, will be a useful benchmark to analyze the performance strengths and weaknesses of each and when they should be used.

An optimal max-cut algorithm may employ many different methods when they will be most useful.

7 Contributions

7.1 Andrew Baumes

Andrew Baumes implemented the CUDA kernel. He wrote the batch script and did testing on AIMOS. He also ran the GSET benchmarks. Lastly, he also contributed to the paper, specifically the CUDA implementation and methodology sections.

7.2 Jeremy Goldberger

Jeremy Goldberger did early research on graphs with solved Max-Cuts, finding the Biq Mac library and retrieving graphs. He wrote a tool to convert files in the rudy-output format to a format accepted by our algorithm. He also contributed to the paper, specifically the "Analysis and Discussion" and "Summary and Future Work" sections.

7.3 Liam Olausson

Liam Olausson supported development of the CUDA kernel by debugging GPU assignment issues as well as performing algorithm analysis and optimization. He also wrote basic scaling Bash scripts as well as working on the paper, he wrote the "Algorithm Design", "Communication Stratagies", "Strong Scaling", and "Weak Scaling" sections as well as a part of the "Analysis and Discussion" section.

7.4 Erik Umble

Erik contributed the MPI portion of the code. He organized the paper's structure and wrote the introduction, mathematical formulation, and parts of the experimental setup. He also found the G-Set benchmark and converted those graphs into the format expected by our algorithm.

References

[1] Enrique Alba and Marco Tomassini. 2002. Parallelism and evolutionary algorithms. *Evolutionary Computation, IEEE Transactions on* 6 (11 2002), 443 – 462. doi:10.1109/TEVC.2002.800880

[2] Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. 2005. A Tutorial on the Cross-Entropy Method. *Annals of Operations Research* 134, 1 (Feb. 2005), 19–67. doi:10.1007/s10479-005-5724-z

[3] Paul Erdos and Alfred Renyi. 1959. On random graphs I. *Publicationes Mathematicae* 6 (1959), 290–297.

[4] Keisuke Fukada. 2022. *max-cut problem benchmark*. https://github.com/0816keisuke/max-cut-problem-benchmark

[5] Michel X. Goemans and David P. Williamson. 1995. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM* 42, 6 (Nov. 1995), 1115–1145. doi:10.1145/227683.227684

[6] Kuk-Hyun Han and Jong-Hwan Kim. 2002. Quantum-inspired evolutionary algorithm for a class of combinatorial optimization. *IEEE Transactions on Evolutionary Computation* 6, 6 (2002), 580–593. doi:10.1109/TEVC.2002.804320

[7] Christoph Helmberg and Franz Rendl. 2000. Gset: Max-Cut benchmark instances. https://web.stanford.edu/~yyye/yyye/Gset/. Instances generated with the RUDY graph generator by Giovanni Rinaldi.

[8] Manuel Laguna, Abraham Duarte, and Rafael Martí. 2009. Hybridizing the cross-entropy method: An application to the max-cut problem. *Computers amp; Operations Research* 36, 2 (Feb. 2009), 487–498. doi:10.1016/j.cor.2007.10.001

[9] L. Lavagna, S. Piperno, A. Ceschini, and M. Panella. 2025. Small graph perturbations, QAOA, and the MaxCut problem. *AVS Quantum Science* 7, 4 (Oct. 2025). doi:10.1116/5.0253160

[10] Franz Rendl, Giovanni Rinaldi, and Angelika Wiegele. 2007. A Branch and Bound Algorithm for Max-Cut Based on Combining Semidefinite and Polyhedral Relaxations. In *Integer Programming and Combinatorial Optimization (IPCO)*. Springer, 295–309.

[11] Reuven Y. Rubinstein. 1997. Optimization of computer simulation models with rare events. *European Journal of Operational Research* 99, 1 (May 1997), 89–112. doi:10.1016/s0377-2217(96)00385-2

[12] Angelika Wiegele. 2007. Biq mac library—a collection of max-cut and quadratic 0-1 programming instances of medium size. *Preprint* 51 (2007), 112–127.